ICS0027 Web application Security – Checkpoint 2
Web based secure password manger

Student: Danila Sisov

Implemented functionality

The Checkpoint 1 scaffold now provides registration, login with a master passphrase, logout and a private vault. Users can add, list, view, edit and delete their own entries. Each entry contains a title, website URL, username, password and optional notes. Flask and Jinja provide a small server-rendered interface; SQLite keeps storage local and straightforward.

Authentication and encrypted storage

Master passphrases are verified using salted Argon2id hashes. A separate random salt derives the user's vault key, so the stored login hash is not the encryption key. Argon2id uses 64 MiB of memory, 3 iterations and parallelism 4 to make password guessing more expensive.

The server encrypts each complete entry JSON payload with AES-256-GCM. A fresh nonce is generated for every save; authenticated additional data binds the ciphertext to its owner, entry ID and format version. This protects stored entry contents and detects tampering. SQLite stores encrypted payloads and account metadata, without plaintext entry fields or vault keys. This implements the design's entry encryption; the whole database file is not encrypted.

Sessions and web protections

Sessions and unlocked vault keys are held in server memory. The pm_session cookie contains only a random token and uses Secure, HttpOnly and SameSite=Strict. Login rotates session and CSRF tokens. Logout, restart and expiry revoke authenticated sessions; limits are 15 minutes idle and one hour absolute.

Every POST requires a session-bound CSRF token. Ownership checks restrict each user to their own entries. Parameterized SQL, input limits, template escaping and security headers reduce injection and browser-side risks. Login throttling and a KDF concurrency limit constrain abuse. Audit events exclude passwords, keys, session tokens and entry contents.

Development setup and verification

Development uses Arch Linux and fish. The application runs at **https://localhost:5443/** with a locally trusted mkcert certificate, providing TLS between the browser and development server. Pinned dependencies and setup, run and test commands are documented in **README.md** and **docs/DEVELOPMENT.md.**

All **24 automated tests passed**. They cover encryption and tampering, storage, authentication, entry operations, isolation between accounts, CSRF, escaping, session rotation and expiry, logout/restart invalidation, request limits and HTTPS checks. Tests use temporary databases. A separate benchmark measures Argon2id verification and key derivation. Work is divided into storage/TLS, authentication/interface, and tests/documentation stages.

Current scope and limitations

This checkpoint uses a single application process for local development. Public deployment with Nginx/Gunicorn remains future work. Sharing, uploads, recovery and MFA are outside this checkpoint. Encryption is server-side: a compromised running server or browser can access unlocked credentials, and Python cannot guarantee complete memory erasure.